
Integration Manual

for MPC560XB ICU Driver

Document Number: IM13ICUASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	9
2.2	Overview.....	10
2.3	About this Manual.....	10
2.4	Acronyms and Definitions.....	11
2.5	Reference List.....	11
Chapter 3		
Building the Driver		
3.1	Build Options.....	13
3.1.1	CW Compiler/Linker/Assembler Options.....	13
3.1.2	DIAB Compiler/Linker/Assembler Options.....	15
3.1.3	GHS Compiler/Linker/Assembler Options.....	17
3.1.4	HT Compiler/Linker/Assembler Options.....	18
3.2	ISR macro.....	19
3.3	Files required for Compilation.....	20
3.4	Setting up the Plugins.....	22
Chapter 4		
Function calls to module		
4.1	Function Calls during Start-up.....	23
4.2	Function Calls during Shutdown.....	23
4.3	Function Calls during Wakeup.....	23
Chapter 5		
Module Requirements		
5.1	Exclusive areas to be defined in BSW scheduler.....	25
5.1.1	ICU_EXCLUSIVE_AREA_00.....	25

Section number	Title	Page
5.1.2	ICU_EXCLUSIVE_AREA_01.....	25
5.1.3	ICU_EXCLUSIVE_AREA_02.....	25
5.1.4	ICU_EXCLUSIVE_AREA_03.....	26
5.1.5	ICU_EXCLUSIVE_AREA_04.....	26
5.1.6	ICU_EXCLUSIVE_AREA_05.....	26
5.1.7	ICU_EXCLUSIVE_AREA_06.....	26
5.1.8	ICU_EXCLUSIVE_AREA_07.....	26
5.1.9	ICU_EXCLUSIVE_AREA_08.....	27
5.1.10	ICU_EXCLUSIVE_AREA_09.....	27
5.1.11	ICU_EXCLUSIVE_AREA_10.....	27
5.1.12	ICU_EXCLUSIVE_AREA_11.....	27
5.1.13	ICU_EXCLUSIVE_AREA_12.....	27
5.1.14	ICU_EXCLUSIVE_AREA_13.....	28
5.1.15	ICU_EXCLUSIVE_AREA_14.....	28
5.1.16	ICU_EXCLUSIVE_AREA_15.....	28
5.1.17	ICU_EXCLUSIVE_AREA_16.....	28
5.1.18	ICU_EXCLUSIVE_AREA_17.....	28
5.1.19	ICU_EXCLUSIVE_AREA_18.....	28
5.1.20	ICU_EXCLUSIVE_AREA_19.....	29
5.1.21	ICU_EXCLUSIVE_AREA_20.....	29
5.1.22	ICU_EXCLUSIVE_AREA_21.....	29
5.2	Critical region exclusivity matrix.....	29
5.3	Peripheral Hardware Requirements.....	30
5.4	ISR to configure within OS – dependencies.....	30
5.5	Other AUTOSAR modules - dependencies.....	33

Chapter 6

Main API Requirements

6.1	Main functions calls within BSW scheduler.....	35
6.2	Calls to notification functions, callbacks, callouts.....	35

Section number	Title	Page
Chapter 7		
Memory Allocation		
7.1	Sections to be defined in MemMap.h.....	37
7.2	Linker command file.....	38

Chapter 8
Configuration Parameter Considerations

Chapter 9
Integration steps



Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	20/01/2012	Syed Hussaini	Updated for MPC560xB BETA 0.9.0 Release
1.1	13/05/2013	Syed Hussaini	1.0.0 Release
1.2	06/June/2014	Hai Pham Minh	1.0.1 Release



Chapter 2

Introduction

This integration manual describes the integration requirements for Autosar ICU Driver for Freescale SemiconductorMPC560XBmicrocontrollers .

The roadmap for the document is as follows:

Building the Driver : This section gives a brief overview of the build procedure (compiler,linker options and source files) and Plugins setup.

Function calls to module : This section lists the various function calls to modules during Start-up, Shutdown and Wake-up.

Module Requirements : This section specifies the various module requirements related to

- Exclusive areas to be defined in BSW scheduler
- Peripheral Hardware Requirements
- Specific interface to other modules
- ISR to configure within OS
- Dependencies with other AUTOSAR modules

Main API Requirements : This section specifies the requirements related to to the main ICU_main API and gives a brief overview of the main functions calls within BSW scheduler, API_Name Requirements and calls to notification functions, callbacks, callouts.

Memory Allocation : This section describes the memory allocation requirements namely the sections to be defined in MemMap.h and the linker command file.

Configuration Parameter Considerations : This section covers the various Pre Compile, Link Time and Post Build time configuration parameters.

Integration steps : This section describes in brief the steps for integrating ICU module.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor:

Table 2-1. MPC560XB Supported Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as **MPC560XB**.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
BSW	Basic Software
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Makefile Interface
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
EcuM	ECU state Manager
GUI	Graphical User Interface
ICU	Input Capture Unit
ISR	Interrupt Sub-Routine
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
OS	Operating System
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
RAM	Random Access Memory
ROM	Read-only Memory
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003ICU Driver Software Specification Document.	4.2.0 Rev 0003
2	MPC5604B Reference Manual	Rev. 8.2, September 2013
3	MPC5607B Reference Manual	Rev. 7.2, May 2012
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
8	Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
9	Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3

Building the Driver

This section describes the source files and various compiler, linker options used for building the Autosar ICUdriver for Freescale SemiconductorMPC560XB microcontrollers. It also explains the Plugins setup procedure.

3.1 Build Options

The ICU driver files are compiled using

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

Note

The TS_T2D13M10I1R0 plugin name is composed as follows:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and
Derivative_Id = 13 identifies the MPC560XB)

3.1.1 CW Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch

Table continues on the next page...

Table 3-2. Assembler Options (continued)

Option	Description
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-3. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 GHS Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-9. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be

Table 3-10. Compiler Options

Option	Description
	added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like --noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,'Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script ie sprig.link. The -c is a synonym of the -T option

3.2 ISR macro

MCAL drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions:

1. OS is not used - AUTOSAR_OS_NOT_USED is defined:

- If USE_SW_VECTOR_MODE is defined:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, drivers' interrupt handlers are normal C functions and the prolog/epilog handle the context save and restore.

- If USE_SW_VECTOR_MODE is not defined:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, drivers' interrupt handlers must save and restore the execution context.

2. Freescale Semiconductor OS is used – AUTOSAR_OS_NOT_USED is not defined

```
#define ISR(IsrName) void OS_isr_##IsrName()
```

In this case, OS is handling the execution context when an interrupt occurs. Drivers' interrupt handlers are normal C functions.

3. Other vendor's OS is used – AUTOSAR_OS_NOT_USED is not defined. Please refer to the OS documentation for description of the ISR macro.

Please refer to the OS documentation for description of the ISR macro.

3.3 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the Autosar ICU driver for Freescale SemiconductorMPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same Autosar major and minor versions can be compiled.

ICU Files:

Icu_TS_T2D13M10I1R0:

- include:
 - a. Icu.h
 - b. Icu_LLD.h
 - c. EMIOS_Icu_LLD.h
 - d. Reg_eSys_EMIOS.h
 - e. Reg_eSys_EMIOS_CfgEx.h
 - f. Reg_eSys_SIUL.h
 - g. Icu_SIUL_LLD.h
 - h. Icu_SIUL_LLD_CfgEx.h
 - i. Reg_eSys_WKPU.h
 - j. Icu_WKPU_LLD.h
 - k. Icu_WKPU_LLD_CfgEx.h
 - l. Dma_LLD.h
 - m. Reg_eSys_DMA.h
- src:
 - Icu.c
 - Icu_LLD.c
 - Icu_EMIOS_LLD.c
 - Icu_EMIOS_LLD_IRQ.c
 - Icu_Reg_eSys_EMIOS.c
 - Icu_SIUL_LLD.c
 - Icu_SIUL_LLD_IRQ.c
 - Icu_WKPU_LLD.c
 - Icu_WKPU_LLD_IRQ.c
 - Icu_Dma_LLD.c

Icu_Cfg.c (For PC Variant) - This file should be generated by the user using a configuration tool for compilation

Icu_PBcfg.c (For PB Variant) - This file should be generated by the user using a configuration tool for compilation

Icu_Cfg.h - This file should be generated by the user using a configuration tool for compilation

Other include files:

- **Files from Base folder:**
 - ..\Base_TS_T2D13M10I1R0include\Compiler.h
 - ..\Base_TS_T2D13M10I1R0include\Compiler_Cfg.h
 - ..\Base_TS_T2D13M10I1R0include\MemMap.h
 - ..\Base_TS_T2D13M10I1R0include\Platform_Types.h

- `..\Base_TS_T2D13M10I1R0\include\Std_Types.h`
- `..\Base_TS_T2D13M10I1R0\include\Reg_eSys.h`
- `..\Base_TS_T2D13M10I1R0\include\Reg_Macros.h`
- **Files from Det folder:**
 - `..\Det_TS_T2D13M10I1R0\include\Det.h`
- **Files from SchM folder:**
 - `..\SchM_TS_T2D13M10I1R0\include\SchM_Icu.h`
- **Files from EcuM folder:**
 - `..\EcuM_TS_T2D13M10I1R0\include\EcuM_Cbk.h`

3.4 Setting up the Plugins

The ICU driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.2.1 (b140128-1223) or later.)

Location of various files inside the plugin folder is explained below.

Module Parameter Definition File:

`..\Icu_TS_T2D13M10I1R0\config\Icu.xdm`

`..\EcuM_TS_T2D13M10I1R0\config\EcuM.xdm`

Code Generation Templates for Pre-Compile time configuration parameters:

`..\Icu_TS_T2D13M10I1R0\generate_PC\src\Icu_Cfg.c`

`..\Icu_TS_T2D13M10I1R0\generate_PC\include\Icu_Cfg.h`

`..\EcuM_TS_T2D13M10I1R0\generate_PC\include\EcuM_Cfg.h`

Code Generation Templates for Post-Build time configuration parameters:

`..\Icu_TS_T2D13M10I1R0\generate_PB\src\Icu_PBcfg.c`

Steps to generate configurations:

1. Copy the module folder (Icu_TS_T2D13M10I1R0) , (EcuM_TS_T2D13M10I1R0) into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the Tresos GUI to modify configuration parameters values
4. Generate the Pre-Compile and Post-Build files.

Chapter 4

Function calls to module

4.1 Function Calls during Start-up

This driver does not need OS Support except for ISRs. Hence can be initialized either in STARTUP1 or STARTUP2 phase of EcuM initialization. This depends on the implementation, desired duration for STARTUP1 & Target hardware design. The API to be called is Icu_Init(ConfigPtr).

Note

For proper driver usage, prior MCU and PORT modules initialization should be done.

4.2 Function Calls during Shutdown

Icu_SetMode(ICU_MODE_SLEEP) API shall be called during GO SLEEP phase of EcuM to configure the hardware for Sleep mode.

4.3 Function Calls during Wakeup

The ICU shall report the wakeup event to EcuM through EcuM_CheckWakeupEvent (event) upon a wakeup event.

Chapter 5

Module Requirements

5.1 Exclusive areas to be defined in BSW scheduler

In the current implementation, ICU is using the services of Schedule Manager (SchM) for entering and exiting the critical regions. SchM implementation is done by the integrators of the MCAL using OS or non-OS services. For testing the ICU, stubs are used for SchM.

All ICU notification functions are called outside any critical region. Global variables updates are performed by ISRs before calling the user notification functions. So the ICU internal state is consistent at the moment of the notification call.

The ISR critical regions must not block the other critical regions to avoid deadlocks. This is ensured by exiting the ISR critical region before calling the user notification functions.

The following critical regions are used in the ICU driver:

5.1.1 ICU_EXCLUSIVE_AREA_00

Used in function Icu_GetTimeElapsed, protects against concurrent access in

- write to Icu_Period
- write to Icu_ActivePulseWidth

5.1.2 ICU_EXCLUSIVE_AREA_01

Used in function Icu_GetDutyCycleValues, protects against concurrent access in

- write to Icu_ActivePulseWidth
- write to Icu_Period

5.1.3 ICU_EXCLUSIVE_AREA_02

Used in function Icu_LLD_GetBitChState, protects against concurrent access in

- read from Icu_ChannelState

5.1.4 ICU_EXCLUSIVE_AREA_03

Used in function Icu_LLD_SetBitChState, protects against concurrent access in

- write to Icu_ChannelState

5.1.5 ICU_EXCLUSIVE_AREA_04

Used in function Icu_LLD_ClearBitChState, protects against concurrent access in

- write to Icu_ChannelState

5.1.6 ICU_EXCLUSIVE_AREA_05

Used in function Icu_LLD_StartTimestamp, protects against concurrent access in

- write to Icu_Buffer
- write to Icu_BufferIndex
- write to Icu_BufferNotify
- write to Icu_BufferSize
- write to Icu_NotifyCount

5.1.7 ICU_EXCLUSIVE_AREA_06

Used in function Icu_LLD_GetTimestampIndex, protects against concurrent access in

- read from Icu_BufferIndex

5.1.8 ICU_EXCLUSIVE_AREA_07

Used in function Icu_LLD_ResetEdgeCount, protects against concurrent access in

- write to Icu_EdgeCount

5.1.9 ICU_EXCLUSIVE_AREA_08

Used in function Icu_LLD_DisableEdgeCount, protects against concurrent access in

- write to Icu_EdgeCount

5.1.10 ICU_EXCLUSIVE_AREA_09

Used in function Icu_LLD_GetEdgeNumbers, protects against concurrent access in

- read from Icu_EdgeCount

5.1.11 ICU_EXCLUSIVE_AREA_10

Used in function Icu_LLD_StartSignalMeasurement, protects against concurrent access in

- write to Icu_ActivePulseWidth
- write to Icu_Period

5.1.12 ICU_EXCLUSIVE_AREA_11

Used in function Icu_LLD_SignalMeasurement, protects against concurrent access in

- write to Icu_ActivePulseWidth
- write to Icu_Period
- write to Icu_Start
- call of Icu_LLD_SetBitChState

5.1.13 ICU_EXCLUSIVE_AREA_12

Used in function Icu_LLD_Timestamp, protects against concurrent access in

- write to Icu_Buffer
- write to Icu_BufferIndex
- read from Icu_BufferNotify
- read from Icu_Buffer_Size
- write to Icu_NotifyCount

5.1.14 ICU_EXCLUSIVE_AREA_13

Used in function Icu_eMIOS_LLD_EnableInterrupt, protects against concurrent access in

- write to ETIMER Reg. STS
- write to EMIOS Reg. EMIOS_CSR
- write to EMIOS Reg. EMIOS_CCR

5.1.15 ICU_EXCLUSIVE_AREA_14

Used in function Icu_eMIOS_LLD_DisableInterrupt, protects against concurrent access in

- write to EMIOS Reg. EMIOS_CCR

5.1.16 ICU_EXCLUSIVE_AREA_15

Used in function Icu_eMIOS_LLD_UCSetMode, protects against concurrent access in

- write to EMIOS Reg. EMIOS_CCR

5.1.17 ICU_EXCLUSIVE_AREA_16

Used in function Icu_eMIOS_LLD_GetInputState, protects against concurrent access in

- write to EMIOS Reg. EMIOS_CSR

5.1.18 ICU_EXCLUSIVE_AREA_17

Used in function Icu_Siul_LLD_EnableInterrupt and Icu_Siul_LLD_DisableInterrupt, protects against concurrent access in

- write to SIUL Reg. ISR
- write to SIUL Reg. IRER

5.1.19 ICU_EXCLUSIVE_AREA_18

Used in function Icu_Siul_LLD_GetInputState, protects against concurrent access in

- write to SIUL Reg. ISR

5.1.20 ICU_EXCLUSIVE_AREA_19

Used in function Icu_WKPU_LLD_EnableInterrupt, protects against concurrent access in

- write to WKPU Reg. WISR
- write to WKPU Reg. IRER
- write to WKPU Reg. WRER

5.1.21 ICU_EXCLUSIVE_AREA_20

Used in function Icu_WKPU_LLD_DisableInterrupt, protects against concurrent access in

- write to WKPU Reg. WISR
- write to WKPU Reg. IRER
- write to WKPU Reg. WRER

5.1.22 ICU_EXCLUSIVE_AREA_21

Used in function Icu_eMIOS_LLD_SetSleepMode and Icu_eMIOS_LLD_SetNormalMode, protects against concurrent access in

- write to EMIOS Reg. EMIOS_CADR
- write to EMIOS Reg. EMIOS_CCNTR

5.2 Critical region exclusivity matrix

Below is the table depicting the exclusivity between different critical region IDs from the ICU driver. If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

	ICU_EXCLUSIVE_AREA_00	ICU_EXCLUSIVE_AREA_01	ICU_EXCLUSIVE_AREA_02	ICU_EXCLUSIVE_AREA_03	ICU_EXCLUSIVE_AREA_04	ICU_EXCLUSIVE_AREA_05	ICU_EXCLUSIVE_AREA_06	ICU_EXCLUSIVE_AREA_07	ICU_EXCLUSIVE_AREA_08	ICU_EXCLUSIVE_AREA_09	ICU_EXCLUSIVE_AREA_10	ICU_EXCLUSIVE_AREA_11	ICU_EXCLUSIVE_AREA_12	ICU_EXCLUSIVE_AREA_13	ICU_EXCLUSIVE_AREA_14	ICU_EXCLUSIVE_AREA_15	ICU_EXCLUSIVE_AREA_16	ICU_EXCLUSIVE_AREA_17	ICU_EXCLUSIVE_AREA_18	ICU_EXCLUSIVE_AREA_19	ICU_EXCLUSIVE_AREA_20
ICU_EXCLUSIVE_AREA_00	x	x																			
ICU_EXCLUSIVE_AREA_01	x	x									x	x									
ICU_EXCLUSIVE_AREA_02				x	x																
ICU_EXCLUSIVE_AREA_03			x	x	x																
ICU_EXCLUSIVE_AREA_04			x	x	x																
ICU_EXCLUSIVE_AREA_05						x	x						x								
ICU_EXCLUSIVE_AREA_06						x							x								
ICU_EXCLUSIVE_AREA_07								x	x	x											
ICU_EXCLUSIVE_AREA_08								x	x	x											
ICU_EXCLUSIVE_AREA_09								x	x												
ICU_EXCLUSIVE_AREA_10	x	x									x	x									
ICU_EXCLUSIVE_AREA_11	x	x									x	x									
ICU_EXCLUSIVE_AREA_12						x	x						x								
ICU_EXCLUSIVE_AREA_13														x	x	x	x				
ICU_EXCLUSIVE_AREA_14														x	x	x					
ICU_EXCLUSIVE_AREA_15														x	x	x					
ICU_EXCLUSIVE_AREA_16														x	x	x	x				
ICU_EXCLUSIVE_AREA_17																		x	x		
ICU_EXCLUSIVE_AREA_18																		x	x		
ICU_EXCLUSIVE_AREA_19																				x	x
ICU_EXCLUSIVE_AREA_20																				x	x

Figure 5-1. Critical region table

5.3 Peripheral Hardware Requirements

EMIOS channels 0-31 for EMIOS0 and EMIOS1 , external interrupts IRQ0–IRQ23 and wakeup interrupts WKUP2–WKUP28 are available for the ICU driver.

Refer Table ICU Hardware Channel availability for MPC560XB family in User Manual

5.4 ISR to configure within OS – dependencies

The following ISR's are used by the ICU driver:

The ISR table is presented below; depending on the derivative used some of the ISR may not be available, for complete details please consult the Reference Manual:

Table 5-1. eMIOS 0 Interrupt Service Routine

EMIOS 0 Interrupts	Hardware interrupt vector
EMIOS_0_CH_0_CH_1_ISR / EMIOS_0_CH_0_ISR / EMIOS_0_CH_1_ISR	141
EMIOS_0_CH_2_CH_3_ISR / EMIOS_0_CH_2_ISR / EMIOS_0_CH_3_ISR	142
EMIOS_0_CH_4_CH_5_ISR / EMIOS_0_CH_4_ISR / EMIOS_0_CH_5_ISR	143
EMIOS_0_CH_6_CH_7_ISR / EMIOS_0_CH_6_ISR / EMIOS_0_CH_7_ISR	144
EMIOS_0_CH_8_CH_9_ISR / EMIOS_0_CH_8_ISR / EMIOS_0_CH_9_ISR	145
EMIOS_0_CH_10_CH_11_ISR / EMIOS_0_CH_10_ISR / EMIOS_0_CH_11_ISR	146
EMIOS_0_CH_12_CH_13_ISR / EMIOS_0_CH_12_ISR / EMIOS_0_CH_13_ISR	147
EMIOS_0_CH_14_CH_15_ISR / EMIOS_0_CH_14_ISR / EMIOS_0_CH_15_ISR	148
EMIOS_0_CH_16_CH_17_ISR EMIOS_0_CH_16_ISR / EMIOS_0_CH_17_ISR	149
EMIOS_0_CH_18_CH_19_ISR EMIOS_0_CH_18_ISR / EMIOS_0_CH_19_ISR	150
EMIOS_0_CH_20_CH_21_ISR EMIOS_0_CH_20_ISR / EMIOS_0_CH_21_ISR	151
EMIOS_0_CH_22_CH_23_ISR EMIOS_0_CH_22_ISR / EMIOS_0_CH_23_ISR	152
EMIOS_0_CH_24_CH_25_ISR EMIOS_0_CH_24_ISR / EMIOS_0_CH_25_ISR	153
EMIOS_0_CH_26_CH_27_ISR EMIOS_0_CH_26_ISR / EMIOS_0_CH_27_ISR	154
EMIOS_0_CH_28_CH_29_ISR EMIOS_0_CH_28_ISR / EMIOS_0_CH_29_ISR	155
EMIOS_0_CH_30_CH_31_ISR EMIOS_0_CH_30_ISR / EMIOS_0_CH_31_ISR	156

Table 5-2. eMIOS 1 Interrupt Service Routine

EMIOS 1 Interrupts	Hardware interrupt vector
EMIOS_1_CH_0_CH_1_ISR / EMIOS_1_CH_0_ISR / EMIOS_1_CH_1_ISR	157
EMIOS_1_CH_2_CH_3_ISR / EMIOS_1_CH_2_ISR / EMIOS_1_CH_3_ISR	158
EMIOS_1_CH_4_CH_5_ISR / EMIOS_1_CH_4_ISR / EMIOS_1_CH_5_ISR	159

Table continues on the next page...

Table 5-2. eMIOS 1 Interrupt Service Routine (continued)

EMIOS 1 Interrupts	Hardware interrupt vector
EMIOS_1_CH_6_CH_7_ISR / EMIOS_1_CH_6_ISR / EMIOS_1_CH_7_ISR	160
EMIOS_1_CH_8_CH_9_ISR / EMIOS_1_CH_8_ISR / EMIOS_1_CH_9_ISR	161
EMIOS_1_CH_10_CH_11_ISR / EMIOS_1_CH_10_ISR / EMIOS_1_CH_11_ISR	162
EMIOS_1_CH_12_CH_13_ISR / EMIOS_1_CH_12_ISR / EMIOS_1_CH_13_ISR	163
EMIOS_1_CH_14_CH_15_ISR / EMIOS_1_CH_14_ISR / EMIOS_1_CH_15_ISR	164
EMIOS_1_CH_16_CH_17_ISR / EMIOS_1_CH_16_ISR / EMIOS_1_CH_17_ISR	165
EMIOS_1_CH_18_CH_19_ISR / EMIOS_1_CH_18_ISR / EMIOS_1_CH_19_ISR	166
EMIOS_1_CH_20_CH_21_ISR / EMIOS_1_CH_20_ISR / EMIOS_1_CH_21_ISR	167
EMIOS_1_CH_22_CH_23_ISR / EMIOS_1_CH_22_ISR / EMIOS_1_CH_23_ISR	168
EMIOS_1_CH_24_CH_25_ISR / EMIOS_1_CH_24_ISR / EMIOS_1_CH_25_ISR	169
EMIOS_1_CH_26_CH_27_ISR / EMIOS_1_CH_26_ISR / EMIOS_1_CH_27_ISR	170
EMIOS_1_CH_28_CH_29_ISR / EMIOS_1_CH_28_ISR / EMIOS_1_CH_29_ISR	171
EMIOS_1_CH_30_CH_31_ISR / EMIOS_1_CH_30_ISR / EMIOS_1_CH_31_ISR	172

Table 5-3. SIUL Interrupt Service Routine

SIUL IRQ Interrupts	Hardware interrupt vector
SIUL_EXT_IRQ_0_7_ISR	41
SIUL_EXT_IRQ_8_15_ISR	42
SIUL_EXT_IRQ_16_23_ISR	43

Table 5-4. WKPU Interrupt Service Routine

Wakeup Unit Interrupts	Hardware interrupt vector
WKPU_EXT_IRQ_0_7_ISR	46
WKPU_EXT_IRQ_8_15_ISR	47
WKPU_EXT_IRQ_16_23_ISR	48
WKPU_EXT_IRQ_24_31_ISR	49

Table 5-5. DMA Interrupts

DMA Interrupts	Hardware interrupt vector
Channel 0	11
Channel 1	12
Channel 2	13
Channel 3	14
Channel 4	15
Channel 5	16
Channel 6	17
Channel 7	18
Channel 8	19
Channel 9	20
Channel 10	21
Channel 11	22
Channel 12	23
Channel 13	24
Channel 14	25
Channel 15	26

NOTE

In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_HW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in hardware vector mode.

5.5 Other AUTOSAR modules - dependencies

Development Error Tracer:

This module is necessary for enabling Development error detection. The API function used is Det_ReportError(). The activation / deactivation of Development error detection is configurable using the 'IcuDevErrorDetect' configuration parameter.

Diagnostic Event Manager:

This module is necessary for enabling reporting of production relevant error status. Since there are no production relevant error codes in ICU this is not used.

ECU State Manager:

This module is used for processing the Wakeup notifications of ICU. Whenever the module is in 'Sleep' mode and a wakeup event occurs on a wakeup capable channel, it is reported to EcuM through the EcuM_CheckWakeupEvent () API. This is configurable using the 'IcuChannelWakeupInfo' configuration parameter.

Configuration dependency to other module:

For generating configuration files of ICU, EcuM also is required as ICU refers to EcuM parameter. EcuM need to be configure first before generating configuration files of ICU.

Hence template files for EcuM are provided at

..\EcuM_<plugin_name>\autosar\EcuM.epd (Module Parameter Definition File – AUTOSAR Format)

..\EcuM_<plugin_name>\config\EcuM.xdm (Module Parameter Definition File – Tresos Format)

Chapter 6

Main API Requirements

6.1 Main functions calls within BSW scheduler

None

6.2 Calls to notification functions, callbacks, callouts

Call-back Notifications:

None.

User Notification:

The ICU Driver provides a notification per channel. The ISR's shall be responsible for resetting the interrupt flags (if needed by hardware) and calling the corresponding notification functions. The notifications can be configured as pointers to user defined functions. If notification is not desired, 'NULL_PTR' shall be configured.

Icu_SignalNotification_<Channel>

The syntax of this function is as follows:

```
void NotificationName
```

```
(  
void  
)
```

According to the last call of Icu_EnableNotification, this notification function shall be called if the requested signal edge (rising / falling / both edges) occurs (once per edge).

Icu_TimestampNotification_<Channel>

The syntax of this function is as follows:

```
void TimestampNotificationName  
(  
void  
)
```

This notification shall be called if the number of requested timestamps (Notification interval > 0) are acquired and if the notification has been enabled by the call of Icu_EnableNotification(). After a call of Icu_DisableNotification() this function must not be called.

An extern declaration of these functions is available in Icu_PBcfg.c. The functions shall be implemented by the user.

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Post Build data:

```
#ifndef ICU_START_CONFIG_DATA_UNSPECIFIED
    #undef ICU_START_CONFIG_DATA_UNSPECIFIED
    #undef MEMMAP_ERROR
    #pragma ghs section const=".pbicu_cfg"
#endif

#ifndef ICU_STOP_CONFIG_DATA_UNSPECIFIED
    #undef ICU_STOP_CONFIG_DATA_UNSPECIFIED
    #undef MEMMAP_ERROR
    #pragma ghs section
#endif
```

For Code:

```
#ifndef ICU_START_SEC_CODE
    #undef ICU_START_SEC_CODE
    #undef MEMMAP_ERROR
    /* no definition -> default compiler settings are used */
#endif
#ifndef ICU_STOP_SEC_CODE
    #undef ICU_STOP_SEC_CODE
    #undef MEMMAP_ERROR
    /* no definition -> default compiler settings are used */
#endif
```

For Variables:

```
#ifndef ICU_START_SEC_VAR_UNSPECIFIED
    #undef ICU_START_SEC_VAR_UNSPECIFIED
    #undef MEMMAP_ERROR
    /* no definition -> default compiler settings are used */
#endif
#ifndef ICU_STOP_SEC_VAR_UNSPECIFIED
```

Linker command file

```
#undef ICU_STOP_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/* no definition -> default compiler settings are used */
#endif
```

For Constant data:

```
#ifdef ICU_START_SEC_CONST_UNSPECIFIED
    #undef ICU_START_SEC_CONST_UNSPECIFIED
    #undef MEMMAP_ERROR
    /* no definition -> default compiler settings are used */
#endif
#ifdef ICU_STOP_SEC_CONST_UNSPECIFIED
    #undef ICU_STOP_SEC_CONST_UNSPECIFIED
    #undef MEMMAP_ERROR
    /* no definition -> default compiler settings are used */
#endif
```

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

Configuration Parameter Considerations

Configuration parameter class for Autosar ICU driver fall into the following variants as defined below:

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
IcuGeneral			
	IcuDevErrorDetect	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuIndex	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuReportWakeupSource	Pre Compile parameter for all Variants of Configuration	Pre Compile
IcuOptionalApis			
	IcuDeInitApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuDisableWakeupApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuEdgeCountApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuEnableWakeupApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuGetDutyCycleValuesApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuGetInputStateApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuGetTimeElapsedApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuGetVersionInfoApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuSetModeApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuTimestampApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuSignalMeasurementApi	Pre Compile parameter for all Variants of Configuration	Pre Compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	IcuWakeupFunctionalityApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuEdgeDetectApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
IcuNonAUTOSAR			
	IcuEnableDualClockMode	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuOverflowNotificationApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuGetInputLevelApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuAllowWkpOnAllChls	Pre Compile parameter for all Variants of Configuration	Pre Compile
IcuConfigSet			
	IcuDmaMaxChannel	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuEXT_ISR_IFCPRDigitalFilter	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuEXT_ISRaltIFCPRDigitalFilter	Pre Compile parameter for all Variants of Configuration	Pre Compile
	IcuMaxChannel	Pre Compile parameter for all Variants of Configuration	Pre Compile
IcuChannel			
	IcuChannelId	VariantPC or VariantPB	Post Build
	IcuDefaultStartEdge	VariantPC or VariantPB	Post Build
	IcuDisableEcumWakeupSourceNotification	VariantPC or VariantPB	Post Build
	IcuHwChannel	VariantPC or VariantPB	Post Build
	IcuWKPU_ISR_WIPUER	VariantPC or VariantPB	Post Build
	IcuEmiosFreeze	VariantPC or VariantPB	Post Build
	IcuEmiosPrescaler	VariantPC or VariantPB	Post Build
	IcuEmiosPrescaler_Alternate	VariantPC or VariantPB	Post Build
	IcuEmiosDigitalFilter	VariantPC or VariantPB	Post Build
	IcuEmiosBusSelect	VariantPC or VariantPB	Post Build
	IcuMeasurementMode	VariantPC or VariantPB	Post Build
	IcuWakeupCapability	VariantPC or VariantPB	Post Build
	IcuDMAEnable	VariantPC or VariantPB	Post Build
	IcuUserModeForDutycycle	VariantPC or VariantPB	Post Build
	IcuOverflowNotification	VariantPC or VariantPB	Post Build
	Icu_EXT_ISR_IFERDigitalFilter	VariantPC or VariantPB	Post Build
	IcuSignalMeasureWithoutInterrupt	VariantPC or VariantPB	Post Build

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	Icu_EXT_ISR_IFMCDigitalFilter	VariantPC or VariantPB	Post Build
IcuSignalEdgeDetection			
	IcuSignalNotification	VariantPC or VariantPB	Post Build
IcuSignalMeasurement			
	IcuSignalMeasurementProperty	VariantPC or VariantPB	Post Build
IcuTimestampMeasurement			
	IcuTimestampMeasurementProperty	VariantPC or VariantPB	Post Build
	IcuTimestampNotification	VariantPC or VariantPB	Post Build
IcuWakeup			
	IcuChannelWakeupInfo	VariantPC or VariantPB	Post Build



Chapter 9

Integration steps

This section gives a brief overview of the steps needed for integrating ICU:

- Generate the required ICU configurations. For more details refer to section [Calls to notification functions, callbacks, callouts](#).
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section [Memory Allocation](#).
- Make sure all include files for compilation are as per section [ISR macro](#).
- Map the ISRs to their vector locations. For more details refer to section [Peripheral Hardware Requirements](#).
- Compile & build the ICU with all the dependent modules. For more details refer to section [Building the Driver and ISR to configure within OS – dependencies](#).



How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2013 Freescale Semiconductor, Inc.

Document Number IM13ICUASR4.0R1.0.1
Revision 1.2